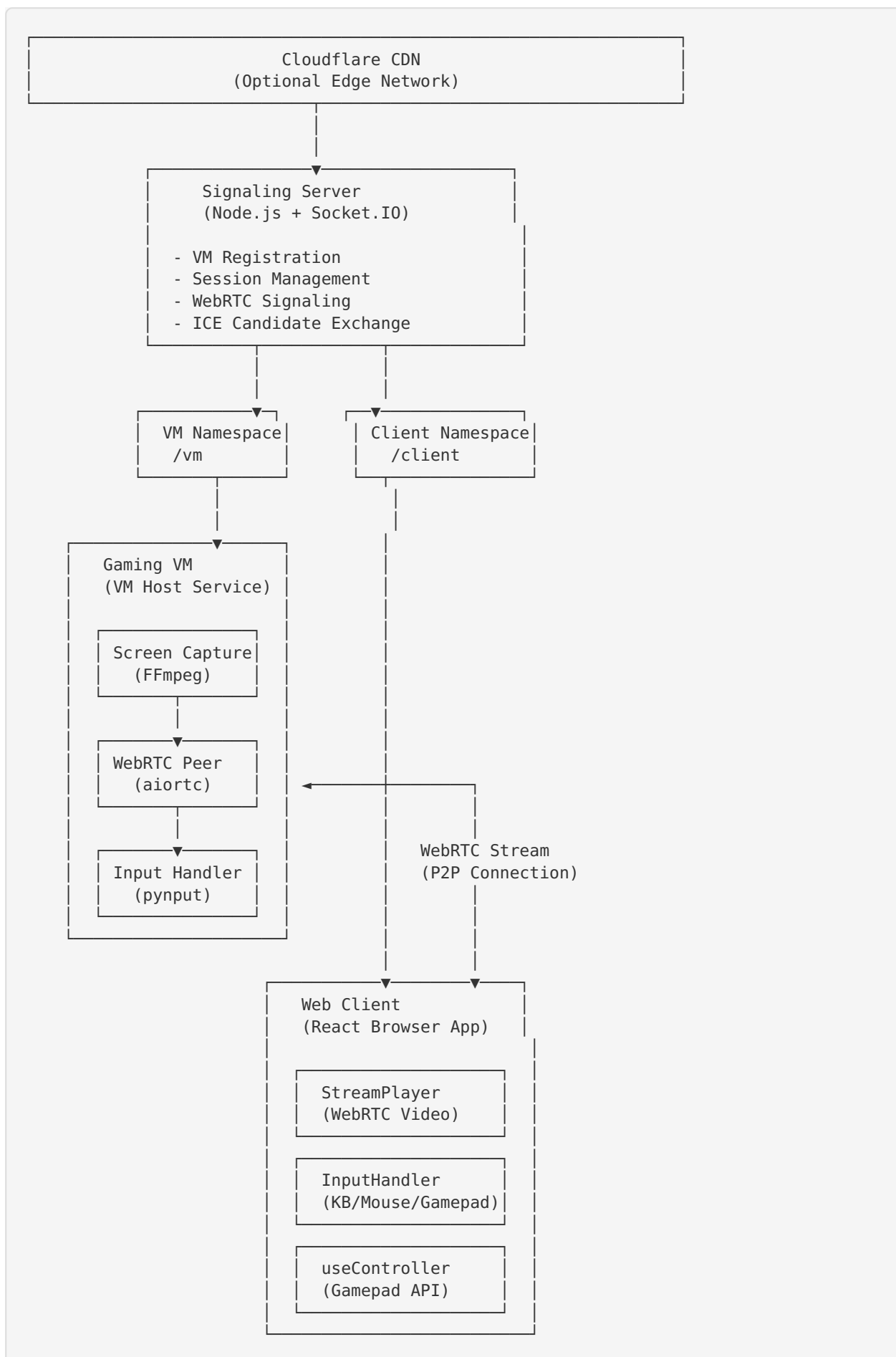


System Architecture

Overview

The cloud gaming infrastructure uses WebRTC for peer-to-peer video streaming with a centralized signaling server for connection setup and session management.

Architecture Diagram



Connection Flow

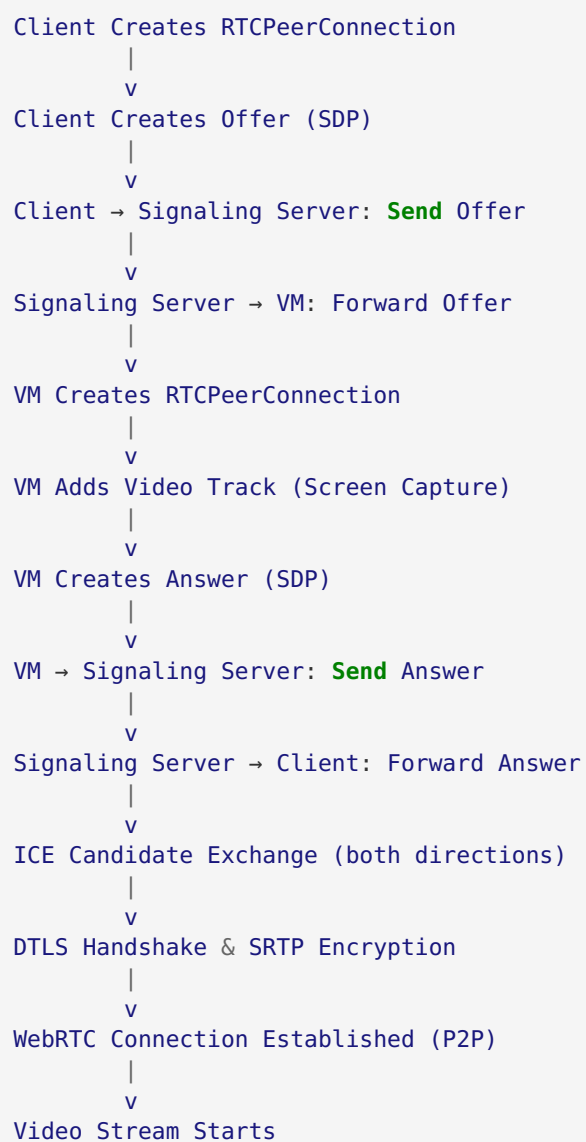
1. VM Startup & Registration (Phone Home)

```
VM Host Service Starts
|
v
Connect to Signaling Server (/vm namespace)
|
v
Send Registration Request
- VM ID
- Hostname
- IP Address
- Capabilities (resolution, bitrate, codecs)
|
v
Signaling Server Generates UUID
|
v
VM Receives Session ID
|
v
Start Heartbeat Loop (every 30s)
|
v
Wait for Client Connection
```

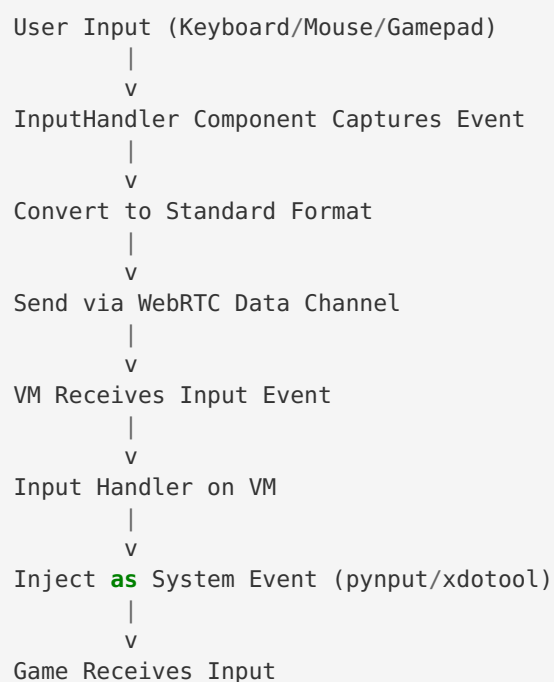
2. Client Connection

```
User Opens Web Client
|
v
Enter Session ID
|
v
Connect to Signaling Server (/client namespace)
|
v
Join Session Request
|
v
Signaling Server Validates Session
|
v
Client Joins Session Room
```

3. WebRTC Connection Establishment



4. Input Flow



Data Channels

Input Channel (Client → VM)

- **Purpose:** Send user inputs to VM
- **Transport:** WebRTC Data Channel (unreliable, no retransmit)
- **Format:** JSON messages
- **Message Types:**
 - `keyboard` : Key press/release
 - `mouse_move` : Mouse movement
 - `mouse_button` : Mouse button press/release
 - `mouse_wheel` : Scroll events
 - `gamepad` : Controller button/axis events

Control Channel (Bi-directional)

- **Purpose:** Quality control and stats
- **Message Types:**
 - `quality_change` : Request quality preset change
 - `stats_request` : Request connection statistics
 - `stats_response` : Connection stats (latency, bitrate, fps)

Session Management

Session Lifecycle

1. **Creation:** VM registers → Server generates UUID
2. **Active:** VM is running and accepting connections
3. **Connected:** Client is connected and streaming

4. **Disconnected:** Client disconnects (VM stays active)
5. **Expired:** VM stops heartbeat or crashes (auto-cleanup)

Session Cleanup

- Heartbeat timeout: 1 hour (configurable)
- Cleanup interval: 1 minute
- Expired sessions automatically removed
- New session ID generated on VM restart

Quality Adaptation

Quality Presets

Preset	Resolution	FPS	Bitrate	Use Case
High	1920x1080	60	8 Mbps	Good networks, low latency
Medium	1280x720	60	4 Mbps	Moderate networks
Low	960x540	60	2 Mbps	Poor networks, high latency

Adaptation Triggers

- Manual: User selects quality preset
- Automatic (future): Based on network stats
- Latency > 100ms → Consider downgrade
- Packet loss > 5% → Downgrade
- Stable connection → Consider upgrade

Security Architecture

Transport Security

1. **Signaling:** WSS (WebSocket Secure) in production
2. **WebRTC Media:** DTLS-SRTP encryption (automatic)
3. **Data Channels:** Encrypted via SCTP over DTLS

Authentication & Authorization

- Session-based: Only clients with valid session ID can connect
- No persistent credentials stored
- Session ID changes on every VM restart
- CORS protection on signaling server

Network Security

- Cloudflare DDoS protection (optional)
- Rate limiting (future)

- IP whitelisting (optional)

Scalability Considerations

Horizontal Scaling

- Multiple signaling servers behind load balancer
- Redis for session state sharing (future)
- Sticky sessions for WebSocket connections

Vertical Scaling

- Signaling Server: Lightweight, handles 1000+ connections per server
- VM Host: Resource-intensive, 1 VM = 1 gaming session

Edge Deployment

- Deploy signaling servers in multiple regions
- Use Cloudflare for global edge network
- Route clients to nearest signaling server

Performance Characteristics

Latency Breakdown

- **Screen Capture:** 1-5ms (FFmpeg)
- **Video Encoding:** 5-15ms (H.264 hardware encoding)
- **Network Transmission:** 10-100ms (varies by location)
- **Video Decoding:** 5-10ms (browser hardware decoding)
- **Display:** 1-2ms

Total End-to-End Latency: 22-132ms (typically 30-60ms on good connections)

Bandwidth Usage

- High Quality: ~8 Mbps
- Medium Quality: ~4 Mbps
- Low Quality: ~2 Mbps
- Overhead: ~100-200 Kbps (signaling, data channel)

Component Dependencies

Signaling Server

```
Express → HTTP Server  
Socket.IO → WebSocket Server  
UUID → Session ID Generation  
Winston → Logging
```

VM Host

FFmpeg → Screen Capture
aiortc → WebRTC Implementation
pynput → Input Injection
socketio-client → Signaling Connection

Web Client

React → UI Framework
Socket.IO Client → Signaling Connection
WebRTC API → Browser Native
Gamepad API → Browser Native